

Foundations of Machine Learning

Assignment 1

Name : Hemachandra

Roll Number : MA25C019

Course Code : DA5400

Question 1

You are given a dataset with data points each in $\mathbb{R}^2$. Also, the dataset contains the class labels (in the third column) which will be used for some supervised learning downstream task.

(a)

Write a piece of code to run the PCA algorithm on this data-set. How much of the variance in the data-set is explained by each of the principal components?

Ans: The given data is $X \in \mathbb{R}^{n \times m}$ where $n = 2$ is the number of features.

The PCA algorithm :

- Center the data: $X = X - \bar{X}$
- Construct the covariance matrix: $C = \frac{1}{m}XX^T$
- Get the eigenvalues of C (in descending order) and the respective eigenvectors: $E, V = \text{Eigensolver}(C)$
- These eigenvectors are the principal components of the data and the eigenvalues are the variance of the data along these principal components

The file '1a.py' contains the code required for this question. I chose to do XX^T since $n = 2 \ll m$ which results in XX^T being a 2×2 matrix, thus less computationally expensive.

The matrix C has two non zero eigenvalues 1.25 and 0.77.

Thus, 61.95% of the variance is explained by the 1st principal component and 38.04% of the variance is explained by the 2nd principal component.

```
61.95483615103681% of Variance in data is explained by the 1th principle component
38.045163848963185% of Variance in data is explained by the 2th principle component
```

Figure 1: Output of code '1a.py'

(b)

Write a piece of code to implement the Kernel PCA algorithm on this dataset. Explore various kernels. For each Kernel, plot the projection of each point in the dataset onto the top-2 principal components. Use one plot for each kernel - In case of RBF kernel, use a different plot for each value of σ that you use.

Ans: If the data does not lie on a linear space PCA may not learn meaning representations hence why we project the data into higher dimension where it might lie in a linear subspace. But this transformation to higher dimension and applying PCA is expensive, thus we opt for kernel PCA that cuts down the computational costs and achieves dimensionality reduction.

The Kernel PCA Algorithm :

- Construct the kernel matrix: $K[i][j] = k(x_i, x_j)$ where $k : \mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R}$ is the kernel function and x_i are the columns of X .
- Center the kernel matrix: $K = K - \frac{1}{m}\mathbb{I}K - \frac{1}{m}K\mathbb{I} + \frac{1}{m^2}\mathbb{I}K\mathbb{I}$ where $\mathbb{I}$ is an $m \times m$ matrix with each element 1.
- Get the eigenvalues of K (in descending order) and the respective eigenvectors: $E, V = \text{Eigensolver}(K)$.
- Normalize the eigenvectors: $v_i = \frac{1}{\sqrt{\lambda_i}}v_i$ where v_i is the eigenvector and λ_i its eigenvalue.
- Get the projection along the principal components: $\alpha_i = Kv_i$ where v_i is the eigenvector.

The file '1b.py' contains the code required by this question. I run the kernel pca algorithm for the kernels $k(x_i, x_j) =$:

- $k_1(p) = (x_i^T x_j + 1)^p$ for $p = 2, 5$ (is a valid kernel as mentioned in class)
- $k_2(\sigma) = e^{-\|x_i - x_j\|^2 / 2\sigma^2}$ for $\sigma = 0.1, 0.5, 1, 2$ (is a valid kernel as mentioned in class)
- $k_1(3) * k_1(0.1)$ is a valid kernel (Product of valid kernels is also valid, 'PRML-Bishop' Chapter-6, section-2, eqn-6.18)
- $k_1(3) + k_2(0.1)$ is a valid kernel (Sum of valid kernels is also valid, 'PRML-Bishop' Chapter-6, section-2, eqn-6.17)

In the code the function K_1 defines the polynomial kernel, K_2 defines the RBF kernel, K_p defines the product of K_1 and K_2 and K_s defines the sum of K_1 and K_2.

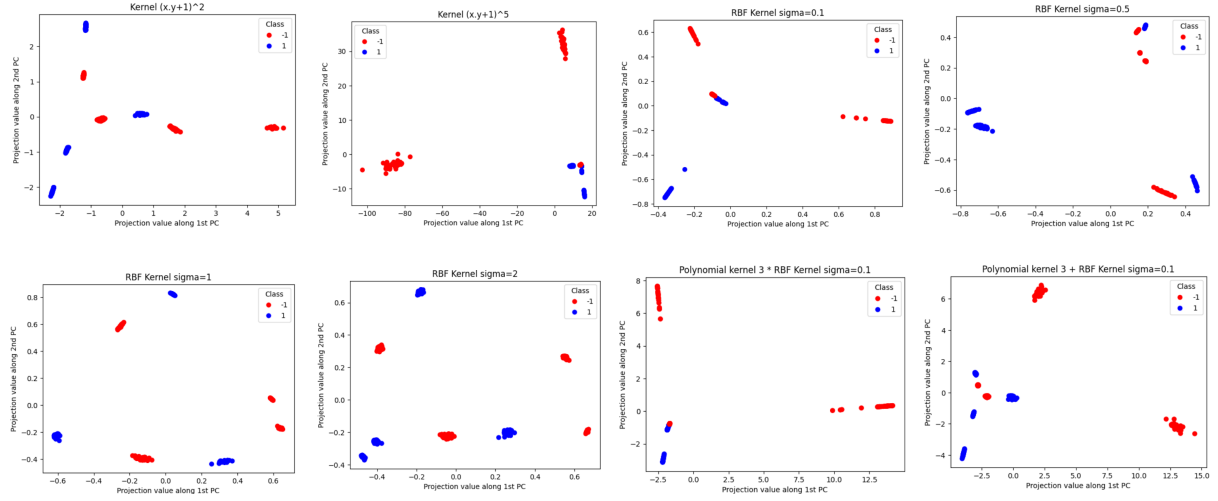


Figure 2: Projections of the data along the top 2 principal components for various kernels

The red color corresponds to the points with label -1 and blue color corresponds to the points with label 1 .

(c)

A linearly separable dataset is one where there exists a vector w such that $\text{sign}(w^T z_i) = y_i$ for all i , where z_i is the datapoint and y_i is the class label and $\text{sign}(u) = 1$ if and only if $u \geq 0$.

Can you find a kernel for the above problem which with just using projections along top-2 principal components will lead to a linearly separable dataset as per the class labels specified?

Ans: Since we use projections only along the top 2 principal components we need to find a line in the 2D space such that the line separates the points with label -1 from the points with label 1 . Looking at the output of 1b.py, the kernel $k_1(3) * k_2(0.1)$ shows a possibility for such a line.

I looked at the grid $[-2, -1.5] \times [-1, -0.5]$ for the projections formed by that kernel and identified that the line passing through $(-1.7, -0.8)$ and $(-1.6, -0.9)$ works for us. By solving the slope-intercept equation for these points we get the line as $y = -x - \frac{5}{2}$. The code '1c.py' plots this line on the 2D space with projections of the data obtained by using the kernel $k_1(3) * k_2(0.1)$.

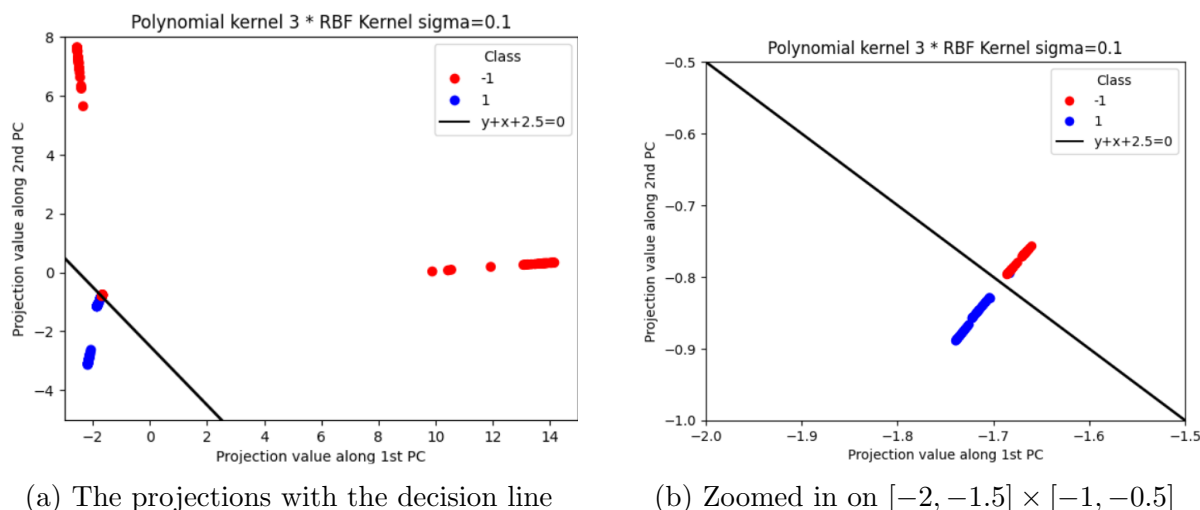


Figure 3: Projections along the top 2 principal components

The code '1c.py' also has an algorithm implemented to check if the line actually separates the classes.

The Algorithm :

- Define the 2×2 confusion matrix cm initially all elements are 0
- Let (p_{i1}, p_{i2}) be the projection of the i th data point with label y_i . Find $f_i = p_{i1} + p_{i2} + 2.5$.
- If $f_i < 0$ predict the class as 1 else -1
- If the prediction is a true negative, add 1 to $cm[0][0]$, if false negative add 1 to $cm[0][1]$, if false positive add 1 to $cm[1][0]$, if true positive add 1 to $cm[1][1]$.

The accuracy of the decision line is 0.9958
 Confusion matrix:
 $\begin{bmatrix} 120. & 1. \\ 0. & 119. \end{bmatrix}$

Figure 4: Accuracy and confusion matrix of the decision line

The output shows that the line $y+x+2.5=0$ almost perfectly forms a decision boundary to the classes with just 1 point lying in the wrong region. Thus the kernel $k_1(3) * k_2(0.1)$, product of polynomial kernel with degree 3 and rbf kernel with $\sigma = 0.1$ leads to a linearly separable dataset.

Question 2

You are given a data-set with 1000 data points each in $\mathbb{R}^2$.

(a)

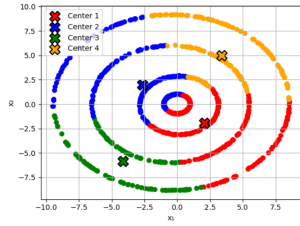
Write a piece of code to run the algorithm studied in class for the K-means problem with $k = 4$. Try 5 different random initialization and plot the error function w.r.t. iterations in each case. In each case, plot the clusters obtained in different colors.

Ans: Given dataset $X \in \mathbb{R}^{2 \times 1000}$, each column of X is a data point with features X_1 and X_2 . The clusters are formed using the Lloyd's algorithm, in which the goal is to minimize the objective function $F(z_0, \dots, z_{999}) = \sum_{i=0}^{999} \|x_i - \mu_{z_i}\|^2$ where z_i is the cluster assigned to the i th data point.

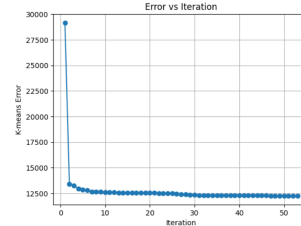
Lloyd's Algorithm :

- Assign each data point uniformly at random to one of the k clusters.
- Do until convergence: Compute the mean of each cluster and assign it as the cluster's center, assign each data point to a cluster whose distance to the center is minimum.
- The algorithm converges when the cluster assignment in the previous iteration is same as the cluster assignment in the current iteration

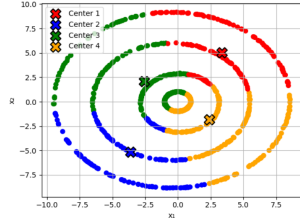
The file '2a.py' contains the code required by this question. The stopping condition implemented is if the cluster initialization Z is equal to the cluster initialization in the previous iteration. A situation might arise when the objective function value in the previous iteration is same as the one in the current iteration, but the cluster assignment might be different and if this assignment happens alternately the algorithm might keep running in the loop. This is prevented by the property of the function *argmin()*, which is used in the list with each entry as the distance between the point and the center of the clusters. This makes it such that if there is a tie in cluster assignment, the cluster with the smallest index is assigned. The error at each iteration is obtained by computing the objective function $F(z_0, \dots, z_{999})$. The code runs the Lloyd's Algorithm 5 times each time for a random initialization of 4 clusters.



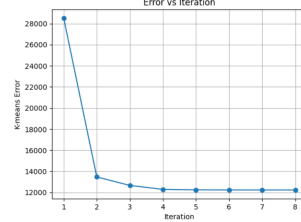
(a) Clusters – initialization 1



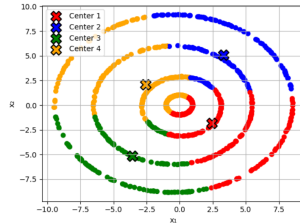
(b) Error vs. iterations – initialization 1



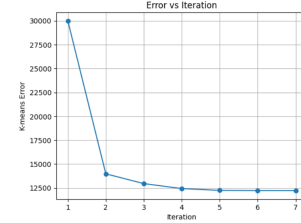
(c) Clusters – initialization 2



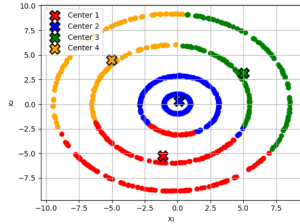
(d) Error vs. iterations – initialization 2



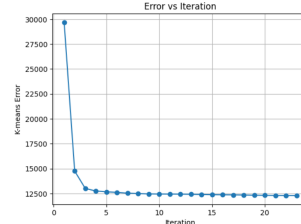
(e) Clusters – initialization 3



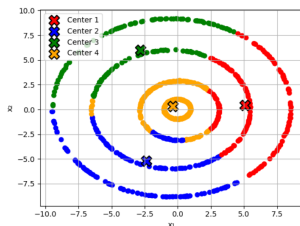
(f) Error vs. iterations – initialization 3



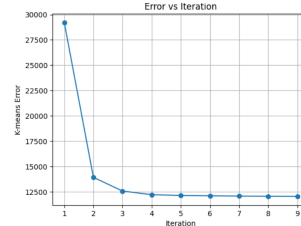
(g) Clusters – initialization 4



(h) Error vs. iterations – initialization 4



(i) Clusters – initialization 5



(j) Error vs. iterations – initialization 5

Figure 5: Clusters formed and Error vs Iterations.

The algorithm converged under 10 iterations for initializations 2,3,5, under 30 for 4 and under 60 for 1. On an average the error at convergence in each case is under 1.25×10^4 .

(b)

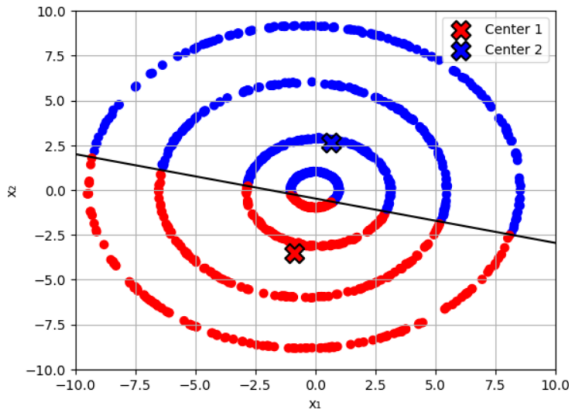
For $K = \{2, 3, 4, 5\}$, fix a random initialization and obtain cluster centers according to K-means algorithm using the fixed initialization. For each value of K , plot the Voronoi regions associated to each cluster center. You can assume the minimum and maximum value in the data-set to be the range for each component of $\mathbb{R}^2$.

Ans: The file '2b.py' has the code required by this question. We need to draw the Voronoi regions after forming the clusters for $k = 2, 3, 4, 5$. To form the clusters we use the same algorithm used to solve question 2a.

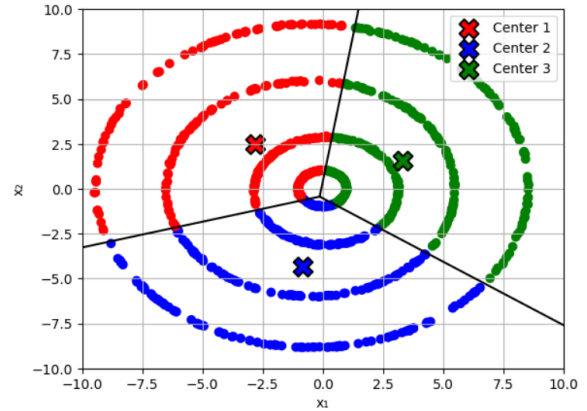
The Algorithm to get Voronoi lines :

- For each pair of cluster centers $\mu_i = (a, b)$ and $\mu_j = (p, q)$ get the line passing through their midpoint and perpendicular to the line joining these centers: $y = mx + c$ where $m = -(\frac{a-p}{b-q})$ and solve for c with the point $(\frac{a+p}{2}, \frac{b+q}{2})$.
- For every other center μ_l , restrict the values of x such that the points on the line y are closer to μ_i and μ_j than to μ_l .

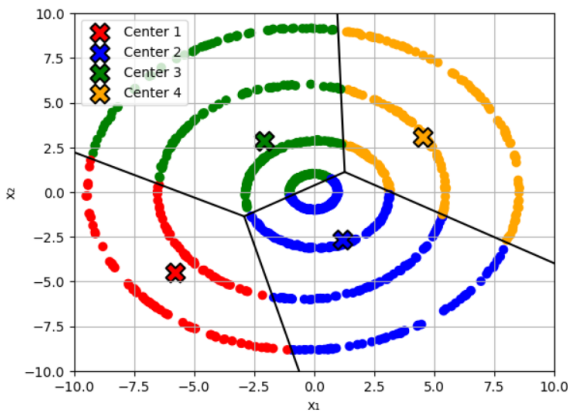
The following images show the clusters and voronoi regions obtained by the code.



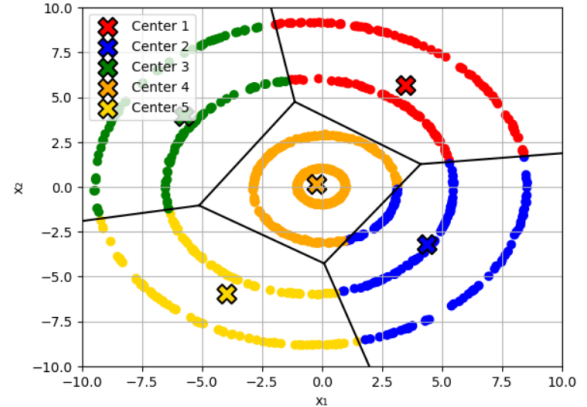
(a) Clusters and Voronoi regions for $K = 2$



(b) Clusters and Voronoi regions for $K = 3$



(c) Clusters and Voronoi regions for $K = 4$



(d) Clusters and Voronoi regions for $K = 5$

Figure 6: Clusters formed and Voronoi regions.

(c)

Run the spectral clustering algorithm (spectral relaxation of K-means using Kernel-PCA) $k = 4$. Choose an appropriate kernel for this data-set and plot the clusters obtained in different colors. Explain your choice of kernel based on the output you obtain.

Ans: The Lloyd's Algorithm always give clusters which are separated by voronoi regions. So if the clusters we expect don't have a geometry like that using Lloyd's won't form those clusters. Hence we go for the kernel clustering method like spectral clustering.
Spectral Clustering Algorithm :

- Get the 1000×1000 kernel matrix: $K[i][j] = k(x_i, x_j)$
- Get Eigenvalues (in descending order) of K and its respective Eigenvectors : $E, V = \text{Eigensolver}(K)$
- Choose the top k eigenvectors of K : $V_{1000 \times k}$
- Normalize the rows of K : $H = \text{RowNormalize}(K)$
- Apply Lloyd's on rows of H
- The cluster assignment obtained for the i th row of H , is the cluster assignment we use for i th data point.

The file '2c.py' contains the required code for this question.

The data points lie on 4 concentric circles. We know that the equation of a circle contains non linear terms of x , upto degree 2. Hence to capture these non linear relationships I use the polynomial kernel of degree 2, $k(x_i, x_j) = (x_i^T x_j + 1)^2$.

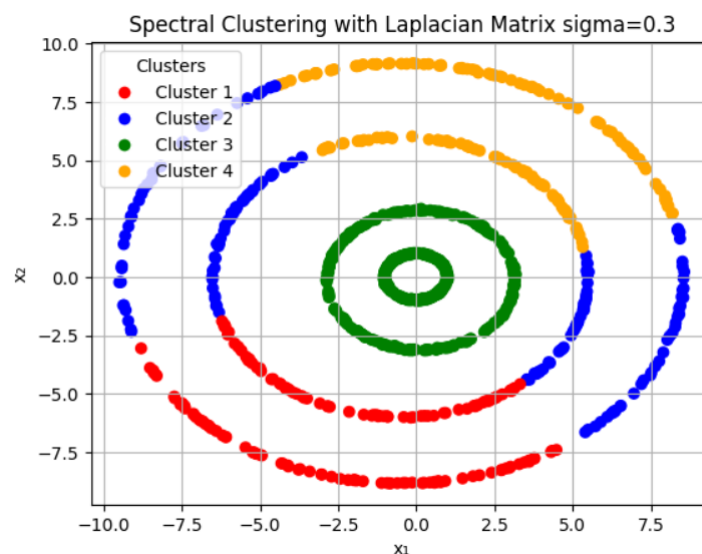


Figure 7: Spectral clustering with polynomial kernel degree 2

But the most intuitive clusters are each circle being one cluster, which is not what we see. Since spectral clustering is almost always synonyms with the Laplacian matrix, we will use it.

Constructing Laplacian matrix :

- Construct the kernel matrix using the RBF kernel: $K[i][j] = e^{-\frac{\|x_i - x_j\|^2}{2\sigma^2}}$
- Define the diagonal matrix D : $D[i][i] = \sum_{j=1}^{1000} K[i][j]$
- Define the Laplacian L : $L = I - D^{-1/2} K D^{-1/2}$

Unlike with kernel matrices where we choose the top k eigenvectors of K , we need to choose the bottom k eigenvectors of L . But since my current implementation of spectral clustering picks the top k eigenvectors, I give $-L$ as the input, which leads to the top k eigenvectors of $-L$ being picked which are actually the bottom k eigenvectors of L .

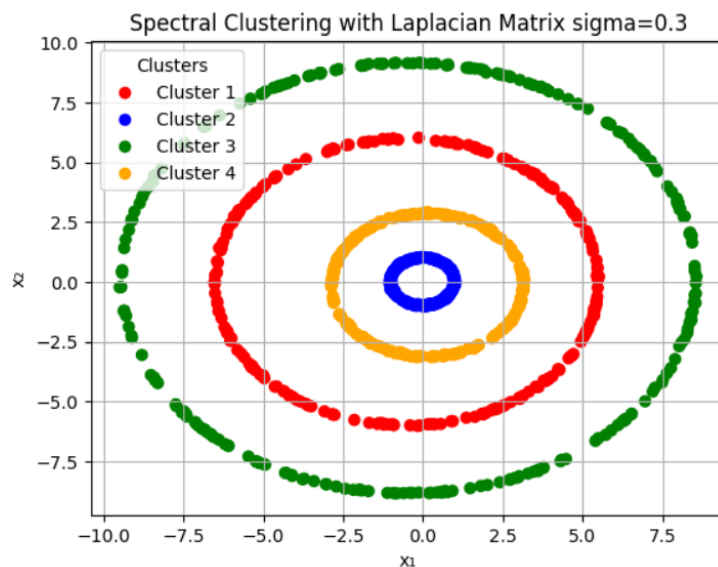


Figure 8: Spectral clustering with Laplacian matrix sigma=0.3

The spectral clustering with Laplacian matrix and $\sigma = 0.3$ gives the most intuitive cluster assignment, where each circle is a cluster.

(d)

Instead of using the method suggested by spectral clustering to map eigenvectors to cluster assignments, use the following method:

Assign data point i to cluster ℓ whenever

$$\ell = \arg \max_{j=1,\dots,k} v_{ji},$$

where $v_j \in \mathbb{R}^n$ is the eigenvector of the Kernel matrix associated with the j -th largest eigenvalue.

How does this mapping perform for this dataset? Explain your insights.

Ans: Since in question 2(c) we got the most intuitive clusters by using the Laplacian matrix with $\sigma = 0.3$ I'll use it instead of a regular kernel for this question. By making the suggested changes to the algorithm we get the clusters as shown below (I still use the top k eigenvectors of $-L$).

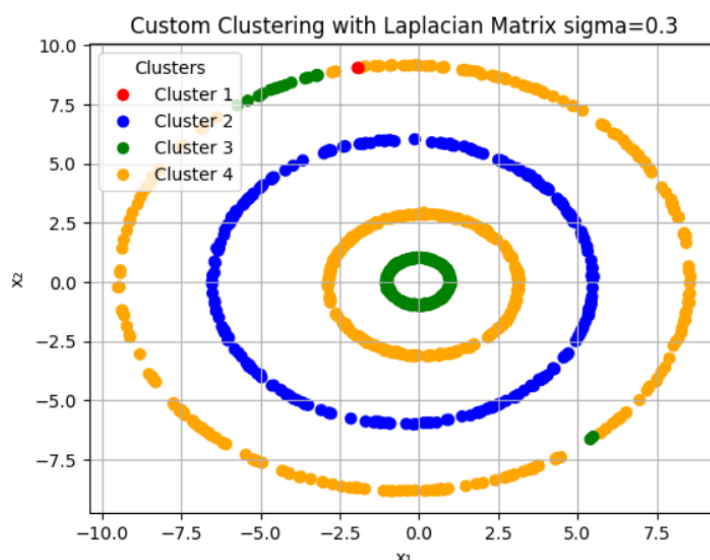


Figure 9: Custom clustering with Laplacian matrix sigma=0.3

The clusters formed are not limited by the voronoi regions. Cluster 1 is almost non-existent in our result. Unlike the regular clustering algorithm we don't get the most intuitive clusters here. Thus the eigenvectors themselves don't carry enough information to do cluster assignment directly.